BÀI 4: BẢO MẬT WEB APP VÀ API

Trong bài học này, chúng ta sẽ khám phá rủi ro của Web App và API, nơi các lỗ hổng như XSS, SQLi, CSRF và SSRF luôn rình rập. Chúng ta sẽ phân tích cơ chế bảo mật của JWT và OAuth 2.0, hiểu rõ cấu trúc token, các điểm yếu thường gặp và cách quản lý vòng đời xác thực. Các bài thực hành sẽ giúp sinh viên trực tiếp xây dựng công cụ quét lỗ hổng, kiểm tra bảo mật token, và tạo lớp bảo vệ API với rate limiting, race condition detection và logging.

4.1 LỖ HỔNG ỨNG DỤNG WEB

4.1.1 XSS (Cross-Site Scripting)

Cross-Site Scripting (XSS) là một lỗ hổng bảo mật xảy ra khi kẻ tấn công chèn mã độc (thường là JavaScript) vào các trang web do người dùng truy cập. Khi được thực thi, mã này có thể đánh cắp cookie, chiếm quyền phiên làm việc, hoặc thực hiện hành vi trái phép thay mặt người dùng. Phân loại XSS:

❖ **Reflected XSS (XSS phản hồi):**

- Mã độc được chèn qua tham số HTTP (URL, form) và phản hồi lại ngay trong nội dung trang web.
- Ví dụ: `http://example.com/search?q=<script>alert('XSS')</script>`
- Nếu máy chủ không mã hóa đúng đầu vào, đoạn mã trên sẽ được thực thi trong trình duyệt nạn nhân.

❖ **Stored XSS (XSS lưu trữ):**

- Payload XSS được lưu vào cơ sở dữ liệu, sau đó hiển thị cho người dùng khác.
- Tình huống phổ biến: Kẻ tấn công gửi bình luận chứa mã độc, mã này sẽ được thực thi mỗi khi người khác xem bình luận đó.

❖ **DOM-based XSS (XSS phía trình duyệt):**

- Lỗ hổng xảy ra hoàn toàn phía client-side, khi JavaScript xử lý dữ liệu từ URL, hash, hoặc DOM mà không kiểm tra đầu vào.
- Ví dụ: `document.write(location.hash);`
- Nếu người dùng truy cập `http://site.com#<script>steal()</script>`, đoạn script sẽ được thực thi.

Hậu quả bảo mật:

- Đánh cắp session token hoặc cookie người dùng.
- Chèn mã độc để điều hướng nạn nhân hoặc tải mã độc khác.
- Tấn công người dùng quản trị để chiếm quyền hệ thống.

Biện pháp phòng chống:

- Mã hóa đầu ra (Output Encoding): Sử dụng thư viện như OWASP Java Encoder hoặc `htmlspecialchars` (PHP).
- Kiểm tra và lọc đầu vào (Input Validation): Từ chối các ký tự đặc biệt không cần thiết.
- CSP (Content Security Policy): giới hạn nguồn script thực thi.
- Tránh sử dụng `innerHTML`, `document.write`, `eval` nếu không cần thiết.

4.1.2 SQL Injection (SQLi) và các biến thể

SQL Injection là một kỹ thuật tấn công cho phép kẻ tấn công chèn hoặc thực thi các câu lệnh SQL độc hại vào truy vấn của ứng dụng, từ đó có thể truy cập, thay đổi, hoặc hủy hoại dữ liệu trong cơ sở dữ liệu.

Cơ chế tấn công: Khi đầu vào từ người dùng được nhúng trực tiếp vào câu lệnh SQL mà không qua kiểm tra hoặc xử lý an toàn, kẻ tấn công có thể "nói chuyện" trực tiếp với cơ sở dữ liệu.

Các biến thể phổ biến:

❖ **Classic SQL Injection (Injection cơ bản):**

- Ví dụ:

```
SELECT * FROM users WHERE username = '$user' AND password = '$pass';
```

- Với đầu vào:

```
' OR 1=1 --
```

- Truy vấn sẽ trở thành:

```
SELECT * FROM users WHERE username = '' OR 1=1 --' AND password = '';
```

- Kết quả: Bỏ qua xác thực, truy cập trái phép.

❖ **Blind SQL Injection (SQL Injection mù):**

- Khi ứng dụng không trả thông báo lỗi nhưng vẫn phản hồi khác nhau dựa trên kết quả truy vấn.
- Kẻ tấn công khai thác bằng cách gửi truy vấn đúng/sai và đoán dần giá trị (ví dụ: bằng cách đo thời gian phản hồi hoặc khác biệt nội dung).

❖ **Time-Based Blind SQLi:**

- Dựa trên việc làm chậm phản hồi nếu truy vấn đúng.
- Ví dụ (MySQL):

```
' OR IF(SUBSTRING(user(),1,1)='r', SLEEP(5), 0) --
```

❖ **Union-Based SQL Injection:**

- Dùng từ khóa UNION để kết hợp kết quả từ nhiều truy vấn.
- Kẻ tấn công có thể trích xuất bảng dữ liệu khác. Ví dụ:

```
' UNION SELECT username, password FROM admin --
```

❖ **Second-Order SQL Injection:**

- Payload được lưu trữ vào hệ thống ở lần đầu, và chỉ thực thi khi nó được sử dụng lại trong truy vấn khác.
- Khó phát hiện vì không tác động tức thì.

4.1.3 CSRF (Cross-Site Request Forgery)

CSRF (tấn công giả mạo yêu cầu từ trang đáng tin) là một hình thức tấn công khiến trình duyệt người dùng thực hiện hành động trái phép trên một ứng dụng web mà người dùng đã đăng nhập, mà không hề hay biết.

Cơ chế tấn công: Kẻ tấn công khai thác quyền truy cập của người dùng hợp lệ, lợi dụng việc trình duyệt tự động gửi cookie phiên khi truy cập trang đích.

Ví dụ kịch bản:

- Người dùng đang đăng nhập ở bank.com với cookie hợp lệ.
- Truy cập 1 trang độc hại do kẻ tấn công tạo, chứa mã:

```
<img src="https://bank.com/transfer?to=hacker&amount=10000">
```

- Trình duyệt tự động gửi request kèm cookie → giao dịch thực hiện mà không cần xác nhận.

Điều kiện để CSRF thành công:

- Phiên đăng nhập còn hiệu lực.
- Trình duyệt gửi tự động cookie/session.
- Ứng dụng không có xác thực bổ sung (như CSRF token).

Biến thể tấn công:

- **GET-based CSRF**: Thực hiện bằng liên kết hình ảnh, script, iframe.
- **POST-based CSRF**: Tạo form ẩn và tự động submit bằng JavaScript.
- **Stored CSRF**: Payload được lưu trữ (hiếm hơn, nguy hiểm hơn).

4.1.4 SSRF (Server-Side Request Forgery)

SSRF là kiểu tấn công trong đó kẻ tấn công lợi dụng khả năng gửi HTTP request từ server, ép máy chủ thực hiện kết nối đến một địa chỉ mà attacker chỉ định, thay vì client.

Cơ chế tấn công:

- Ứng dụng cho phép người dùng chỉ định URL hoặc tài nguyên từ xa (ví dụ: tính năng "fetch ảnh từ URL"). Nếu server không kiểm soát đầu vào, attacker có thể đánh lừa server gửi request đến nội bộ, ví dụ:

```
GET /fetch?url=http://127.0.0.1:8080/admin
```

- Từ đó, attacker có thể:
  - Truy cập dịch vụ nội bộ không public.

- Đọc metadata cloud (AWS, GCP, etc.).
- Tấn công từ bên trong (pivot attack).

Ví dụ:

```
curl 'https://vulnerable-site.com/load?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/'
```

→ Truy xuất thông tin IAM của máy chủ EC2 trên AWS. Nguy hiểm cấp độ chiếm quyền root trên cloud.

Các biến thể:

- **Basic SSRF**: Truy cập IP nội bộ như localhost, 127.0.0.1, 169.254.x.x.
- **Blind SSRF**: Không thấy phản hồi trực tiếp → dùng DNS logging hoặc đo thời gian để xác nhận.
- **Out-of-Band SSRF (OOB)**: Payload tạo phản hồi qua hệ thống thứ ba, ví dụ: DNS hoặc webhook.

4.2 THỰC HÀNH: WEBVULNSCANNER

4.2.1 Yêu cầu

Xây dựng công cụ *Web Vulnerability Scanner* có các chức năng:

- *XSSScanner*: Phát hiện lỗ hổng XSS thông qua thư viện các payload đã được chuẩn hóa.
- *SQLInjectionTester*: Kiểm thử các biến thể tấn công SQL Injection trên nhiều hệ quản trị cơ sở dữ liệu khác nhau.
- *CSRFDetector*: Phân tích và kiểm tra khả năng bảo vệ CSRF của ứng dụng.
- *SSRFTester*: Phát hiện lỗ hổng SSRF và các endpoint tiềm ẩn nguy cơ truy cập nội bộ.
- *ReportGenerator*: Tạo báo cáo chi tiết về các lỗ hổng phát hiện được, bao gồm mô tả, mức độ rủi ro và gợi ý khắc phục.

4.2.2 Hướng dẫn thực hành

- Tạo folder "`web-vuln-scanner`" trong folder source code clone về từ Github.

- Trong folder này, tạo các folder "report", "scanners", "tests", "ui" và các file "__init__.py", "main.py", "requirements.txt".
- Code cho file "requirements.txt":

requirements.txt U

web-vuln-scanner > requirements.txt

```
1  Flask
```

- Cài các gói cần thiết bằng lệnh pip:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\web-vuln-scanner> pip install -r .\requirements.txt
Requirement already satisfied: Flask in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from -r .\requiremer
Requirement already satisfied: Werkzeug>=2.3.7 in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from Flask
Requirement already satisfied: Jinja2>=3.1.2 in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from Flask->
Requirement already satisfied: itsdangerous>=2.1.2 in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from F
Requirement already satisfied: click>=8.1.3 in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from Flask->-
Requirement already satisfied: blinker>=1.6.2 in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from Flask-
Requirement already satisfied: colorama in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from click>=8.1.3
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\hongphuoc\appdata\local\programs\python\python313\lib\site-packages (from Jinja
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\web-vuln-scanner>
```

- File "__init__.py" không cần code.
- Tạo file "scanners/__init__.py".
- Code cho file "scanners/csrf_detector.py":

web-vuln-scanner > scanners > csrf_detector.py > ...

```python
import re

class CSRFDetector:
    csrf_token_patterns = [
        re.compile(r'name=["\']csrf_token["\']', re.I),
        re.compile(r'name=["\']_csrf["\']', re.I),
        re.compile(r'name=["\']csrfmiddlewaretoken["\']', re.I)
    ]

    def scan(self, html_content):
        for pattern in self.csrf_token_patterns:
            if pattern.search(html_content):
                return {'csrf_protected': True}
        return {'csrf_protected': False, 'desc':
            'No CSRF token found, vulnerable to CSRF', 'risk': 'Medium'}

```

- Code cho file "scanners/sql_injection_tester.py":

web-vuln-scanner > scanners > sql_injection_tester.py > ...

```python
from urllib.parse import urlparse, parse_qs

class SQLInjectionTester:
    payloads = [
        "' OR '1'='1",
        "'; DROP TABLE users;--",
        "\" OR 1=1--",
        "' UNION SELECT NULL--"
    ]

    def scan(self, url):
        findings = []
        parsed = urlparse(url)
        params = parse_qs(parsed.query)

        for param, values in params.items():
            for val in values:
                for payload in self.payloads:
                    if payload in val:
                        findings.append({
                            'param': param,
                            'payload': payload,
                            'desc': 'Potential SQL Injection',
                            'risk': 'Critical'
                        })
        return findings

```

- Code cho file "`scanners/ssrf_tester.py`":

web-vuln-scanner > scanners > ssrf_tester.py > SSRFTester

```python
import re
from urllib.parse import urlparse, parse_qs

class SSRFTester:
    internal_ips = [
        re.compile(r'127\.0\.0\.1'),
        re.compile(r'localhost'),
        re.compile(r'10\.\d{1,3}\.\d{1,3}\.\d{1,3}'),
        re.compile(r'192\.168\.\d{1,3}\.\d{1,3}'),
        re.compile(r'172\.(1[6-9]|2\d|3[0-1])\.\d{1,3}\.\d{1,3}')
    ]

    def scan(self, url):
        findings = []
        parsed = urlparse(url)
        params = parse_qs(parsed.query)

        for param, values in params.items():
            for val in values:
                for pattern in self.internal_ips:
                    if pattern.search(val):
                        findings.append({
                            'param': param,
                            'value': val,
                            'desc': 'Potential SSRF to internal resource',
                            'risk': 'High'
                        })
        return findings

```

- Code cho file "`scanners/xss_scanner.py`":

web-vuln-scanner > scanners > xss_scanner.py > ...

```python
import requests

class XSSScanner:
    def __init__(self):
        self.payloads = ["<script>alert(1)</script>",
                         "'><svg/onload=alert(1)>"]

```

```
    def scan(self, url):
        results = []
        for payload in self.payloads:
            test_url = f"{url}?q={payload}"
            try:
                resp = requests.get(test_url, timeout=5)
                if payload in resp.text:
                    results.append({
                        "type": "XSS",
                        "payload": payload,
                        "url": test_url,
                        "risk": "HIGH"
                    })
            except Exception as e:
                pass
        return results

```

- Tạo file "`tests/__init__.py`".

- Code cho file "`tests/test_csrf.py`":

web-vuln-scanner > tests > test_csrf.py > ...

```
import unittest
from scanners.csrf_detector import CSRFDetector

class TestCSRFDetector(unittest.TestCase):
    def setUp(self):
        self.detector = CSRFDetector()

    def test_csrf_token_present(self):
        html = '''<form><input type="hidden"
                    name="csrf_token" value="abc123"/></form>'''
        result = self.detector.scan(html)
        self.assertTrue(result.get('csrf_protected'))

    def test_csrf_token_missing(self):
        html = '<form><input type="text" name="username"/></form>'
        result = self.detector.scan(html)
        self.assertFalse(result.get('csrf_protected'))
        self.assertIn('vulnerable', result.get('desc').lower())

if __name__ == '__main__':
    unittest.main()

```

- Code cho file "`tests/test_sql_injection.py`":

web-vuln-scanner > tests > test_sql_injection.py > ...

```python
import unittest
from scanners.sql_injection_tester import SQLInjectionTester

class TestSQLInjectionTester(unittest.TestCase):
    def setUp(self):
        self.scanner = SQLInjectionTester()

    def test_detect_sql_injection(self):
        url = "http://example.com/page?id=' OR '1'='1"
        findings = self.scanner.scan(url)
        self.assertTrue(any(f['param'] == 'id' and 'SQL Injection'
                            in f['desc'] for f in findings))

    def test_no_vulnerability(self):
        url = "http://example.com/page?id=normalvalue"
        findings = self.scanner.scan(url)
        self.assertEqual(findings, [])

if __name__ == '__main__':
    unittest.main()

```

- Code cho file "`tests/test_ssrf.py`":

web-vuln-scanner > tests > test_ssrf.py > ...

```python
import unittest
from scanners.ssrf_tester import SSRFTester

class TestSSRFTester(unittest.TestCase):
    def setUp(self):
        self.tester = SSRFTester()

    def test_detect_ssrf(self):
        url = "http://example.com/fetch?url=http://127.0.0.1/admin"
        findings = self.tester.scan(url)
        self.assertTrue(any(f['param'] == 'url' and 'SSRF'
                            in f['desc'] for f in findings))

    def test_no_ssrf(self):
        url = "http://example.com/fetch?url=http://example.com"
        findings = self.tester.scan(url)
        self.assertEqual(findings, [])

if __name__ == '__main__':
    unittest.main()

```

- Code cho file "`tests/test_xss.py`":

web-vuln-scanner > tests > test_xss.py > ...

```python
import unittest
from unittest.mock import patch
from scanners.xss_scanner import XSSScanner

class TestXSSScanner(unittest.TestCase):
    def setUp(self):
        self.scanner = XSSScanner()

    @patch('scanners.xss_scanner.requests.get')
    def test_detect_xss(self, mock_get):
        mock_get.return_value.text = '''
                <html><body><script>alert(1)</script></body></html>'''

        url = "http://example.com"
        findings = self.scanner.scan(url)

        self.assertTrue(any(f['payload'] == "<script>alert(1)</script>"
                            for f in findings))
        self.assertEqual(findings[0]['type'], 'XSS')
        self.assertEqual(findings[0]['risk'], 'HIGH')

if __name__ == '__main__':
    unittest.main()

```

- Code cho file "`tests/ test_report.py`":

web-vuln-scanner > tests > test_report.py > ...

```python
import unittest
from report.report_generator import ReportGenerator

class TestReportGenerator(unittest.TestCase):
    def setUp(self):
        self.generator = ReportGenerator()

    def test_generate_report_with_list(self):
        findings_list = [
            [{'param': 'input', 'desc': 'XSS found', 'risk': 'High'}],
            [{'param': 'id', 'desc': 'SQL Injection', 'risk': 'Critical'}]
        ]
```

```
        report = self.generator.generate(findings_list)
        self.assertEqual(len(report['vulnerabilities']), 2)
        self.assertTrue(any(v['desc'] == 'XSS found'
                            for v in report['vulnerabilities']))

    def test_generate_report_with_dict(self):
        findings_dict = {'param': 'url', 'desc': 'SSRF found', 'risk': 'High'}
        report = self.generator.generate([findings_dict])
        self.assertEqual(len(report['vulnerabilities']), 1)
        self.assertEqual(report['vulnerabilities'][0]['param'], 'url')

if __name__ == '__main__':
    unittest.main()

```

- Code cho file "`main.py`":

web-vuln-scanner > main.py > ...

```python
from scanners.xss_scanner import XSSScanner
from scanners.sql_injection_tester import SQLInjectionTester
from scanners.csrf_detector import CSRFDetector
from scanners.ssrf_tester import SSRFTester
from report.report_generator import ReportGenerator

def main():
    url = '''http://example.com/page?
            input=<script>alert(1)</script>&id='
            OR '1'='1&url=http://127.0.0.1/admin'''
    html = '''<form><input type="hidden"
            name="csrf_token" value="abc123"/></form>'''

    xss = XSSScanner()
    sql = SQLInjectionTester()
    csrf = CSRFDetector()
    ssrf = SSRFTester()
    report_gen = ReportGenerator()

```

```
    findings = []
    findings.append(xss.scan(url))
    findings.append(sql.scan(url))
    findings.append(csrf.scan(html))
    findings.append(ssrf.scan(url))

    report = report_gen.generate(findings)
    print("=== Vulnerability Report ===")
    for vuln in report['vulnerabilities']:
        print(f'''Param: {vuln.get('param', 'N/A')}, Desc: {vuln.get('desc')},
              Risk: {vuln.get('risk')}''')

if __name__ == '__main__':
    main()

```

- Tạo file "`report/__init__.py`".

- Code cho file "`report/report_generator.py`":

web-vuln-scanner > report > report_generator.py > ReportGenerator > generate

```
class ReportGenerator:
    def generate(self, findings_list):
        report = {'vulnerabilities': []}
        for findings in findings_list:
            if isinstance(findings, list):
                report['vulnerabilities'].extend(findings)
            elif isinstance(findings, dict):
                report['vulnerabilities'].append(findings)
        return report

```

- Code cho file "`ui/app.py`":

web-vuln-scanner > ui > app.py > ...

```
import sys
import os
sys.path.insert(0, os.path.abspath(os.path.join(
    os.path.dirname(__file__), '..')))

from flask import Flask, render_template, request
from scanners.xss_scanner import XSSScanner
from scanners.sql_injection_tester import SQLInjectionTester
from scanners.csrf_detector import CSRFDetector
from scanners.ssrf_tester import SSRFTester
from report.report_generator import ReportGenerator

app = Flask(__name__)

```

web-vuln-scanner > ui > app.py > ...

```python
@app.route("/", methods=["GET", "POST"])
def index():
    if request.method == "POST":
        url = request.form.get("url")
        options = request.form.getlist("scanner")
        results = []

        if "xss" in options:
            xss = XSSScanner()
            results.append(xss.scan(url))

        if "sql" in options:
            sql = SQLInjectionTester()
            results.append(sql.scan(url))

        if "csrf" in options:
            csrf = CSRFDetector()
            html = "<html><form></form></html>"
            results.append([csrf.scan(html)])

        if "ssrf" in options:
            ssrf = SSRFTester()
            results.append(ssrf.scan(url))

        report = ReportGenerator().generate(results)
        return render_template("result.html", report=report)

    return render_template("index.html")

if __name__ == "__main__":
    app.run(debug=True)

```

- Code cho file "`ui/templates/index.html`":

web-vuln-scanner > ui > templates > index.html

```html
<!DOCTYPE html>
<html>
<head>
    <title>Web Vulnerability Scanner</title>
    <style>
        body { font-family: Arial; margin: 2em; }
        input, button { padding: 0.5em; margin: 0.5em 0; width: 100%; }
    </style>
</head>
```

web-vuln-scanner > ui > templates > <> index.html

```
<body>
    <h2>Web Vulnerability Scanner</h2>
    <form method="POST">
        <label>URL cần kiểm thử:</label>
        <input type="text" name="url" required placeholder="http://example.com"/>

        <label>Chọn công cụ kiểm thử:</label>
        <input type="checkbox" name="scanner" value="xss" checked> XSS Scanner<br>
        <input type="checkbox" name="scanner" value="sql" checked> SQL Injection Tester<br>
        <input type="checkbox" name="scanner" value="csrf" checked> CSRF Detector<br>
        <input type="checkbox" name="scanner" value="ssrf" checked> SSRF Tester<br>

        <button type="submit">Bắt đầu kiểm thử</button>
    </form>
</body>
</html>

```

- Code cho file "`ui/templates/result.html`":

web-vuln-scanner > ui > templates > <> result.html

```
<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <title>Kết quả kiểm thử</title>
    <style>
        body { font-family: "Segoe UI", Arial, sans-serif;
               margin: 2em; background: #f9f9f9; color: #222; }
        .vuln {
            padding: 1em;
            border-radius: 8px;
            margin-bottom: 1em;
            border-left: 6px solid;
        }
        .HIGH { background: #ffeaea; border-color: #d00; }
        .MEDIUM { background: #fff6e6; border-color: #e69500; }
        .LOW { background: #eaffea; border-color: #0b0; }
        .title { font-size: 1.3em; margin-bottom: 0.5em; }
        .label { font-weight: bold; }
        a { text-decoration: none; color: #007BFF; }
    </style>
</head>
<body>
    <h2>Báo cáo lỗ hổng</h2>

```

web-vuln-scanner > ui > templates > <> result.html

```
<body>
    {% if report.vulnerabilities %}
        {% for v in report.vulnerabilities %}
            <div class="vuln {{ v.risk | default('LOW') }}">
                <div class="title">{{ v.type |
                    default('Lỗ hổng chưa xác định') }}</div>
                <div><span class="label">Vị trí:</span> {{ v.param |
                    default('Không rõ') }}</div>
                <div><span class="label">Mô tả:</span> {{ v.desc |
                    default('Chưa có mô tả') }}</div>
                <div><span class="label">Mức độ rủi ro:</span> {{ v.risk |
                    default('LOW') }}</div>
                {% if v.url %}
                    <div><span class="label">URL:</span>
                        <a href="{{ v.url }}" target="_blank">{{ v.url }}</a></div>
                {% endif %}
                {% if v.payload %}
                    <div><span class="label">Payload:</span>
                        <code>{{ v.payload }}</code></div>
                {% endif %}
            </div>
        {% endfor %}
    {% else %}
        <p>Không phát hiện lỗ hổng nào.</p>
    {% endif %}

    <p><a href="/">Quay lại trang kiểm thử</a></p>
</body>
</html>
```

- Chạy và kiểm tra unittest:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\web-vuln-scanner> python -m unittest discover tests
.........
----------------------------------------------------------------------
Ran 9 tests in 0.002s

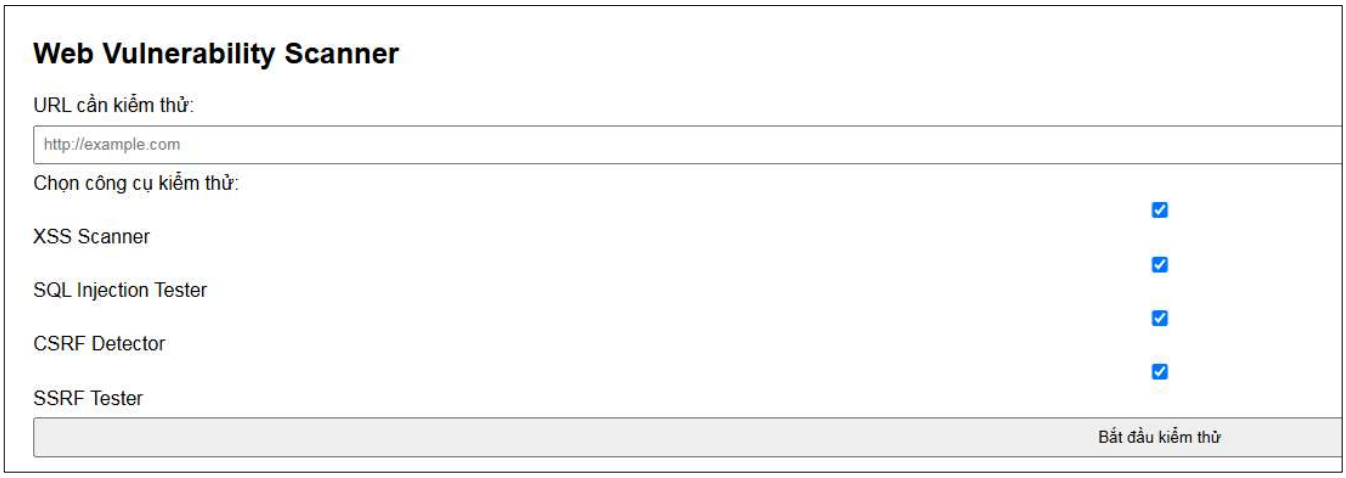
OK
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\web-vuln-scanner>
```

- Chạy kiểm tra chương trình chính:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\web-vuln-scanner> python .\main.py
=== Vulnerability Report ===
Param: N/A, Desc: None,
             Risk: None
Param: url, Desc: Potential SSRF to internal resource,
             Risk: High
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\web-vuln-scanner>
```

- Chạy và kiểm tra web UI:





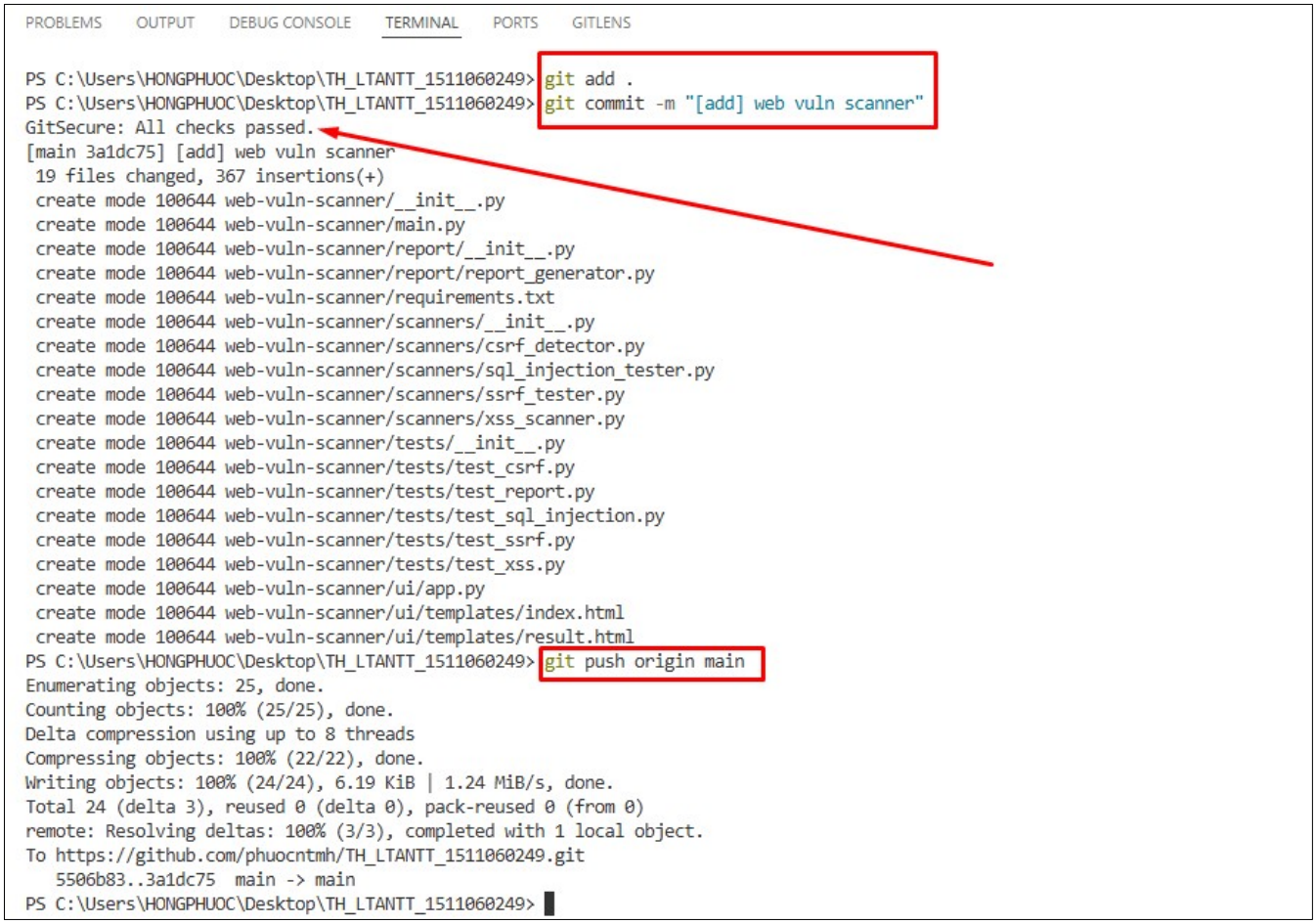
- Kết quả quét lỗ hổng:



- Có thể dùng các trang sau để kiểm thử:

```
https://xss-game.appspot.com/level1
http://testphp.vulnweb.com/artists.php?artist=1'
http://testphp.vulnweb.com/showimage.php?file=1
' OR '1'='1
" OR "1"="1
1'; --
https://webhook.site => Lấy URL ngẫu nhiên.
```

- Commit code, push các thay đổi lên remote repo:



4.3 JWT VÀ BẢO MẬT OAUTH

4.3.1 Cấu trúc và Lỗ hổng của JWT

JWT (JSON Web Token) là một chuẩn mã hóa nhẹ dùng để truyền tải thông tin xác thực giữa các hệ thống một cách an toàn. Mỗi token là một chuỗi văn bản mã hóa theo Base64, không trạng thái (stateless), có thể kiểm tra tính toàn vẹn thông qua chữ ký số.

Cấu trúc của JWT: Một JWT gồm 3 phần chính, phân cách bằng dấu chấm (.):

```
<Header>.<Payload>.<Signature>
```

1. Header:

Khai báo thuật toán ký và loại token, ví dụ:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2. Payload:

Chứa thông tin (claims), ví dụ:

```
{
  "sub": "user123",
  "role": "admin",
  "exp": 1719820800
}
```

Các claim phổ biến:

- sub: Subject (định danh người dùng).
- iat: Issued at (thời điểm phát hành).
- exp: Expiration time (hết hạn).
- iss: Issuer (bên phát hành).

3. Signature:

Dùng để xác minh tính toàn vẹn của token. Ví dụ với HMAC SHA-256:

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  secret
)
```

Các lỗ hổng phổ biến trong JWT:

Lỗ hổng	Mô tả
alg=none	Một số JWT library chấp nhận alg=none, cho

	phép attacker bỏ qua chữ ký.
Key confusion (HS256 vs RS256)	Chuyển từ asymmetric (RS256) sang symmetric (HS256) để giả mạo token bằng public key.
Brute force secret	Nếu khóa bí mật quá yếu, có thể bị dò bằng từ điển.
Token replay	Dùng lại token đã hết hạn hoặc đánh cắp từ trình duyệt.
Lỗi trong xác thực thời gian (exp, iat)	Không kiểm tra kỹ exp → cho phép token dùng vô hạn.
Thông tin nhạy cảm trong Payload	Vì payload không mã hóa (chỉ base64) → có thể đọc được nếu bị đánh cắp.

4.3.2 OAuth 2.0 Flows và Các vấn đề bảo mật

OAuth 2.0 là giao thức ủy quyền truy cập cho phép ứng dụng bên thứ ba truy cập vào tài nguyên người dùng mà không cần chia sẻ mật khẩu. Thay vào đó, hệ thống sử dụng các access token được cấp sau quá trình ủy quyền.

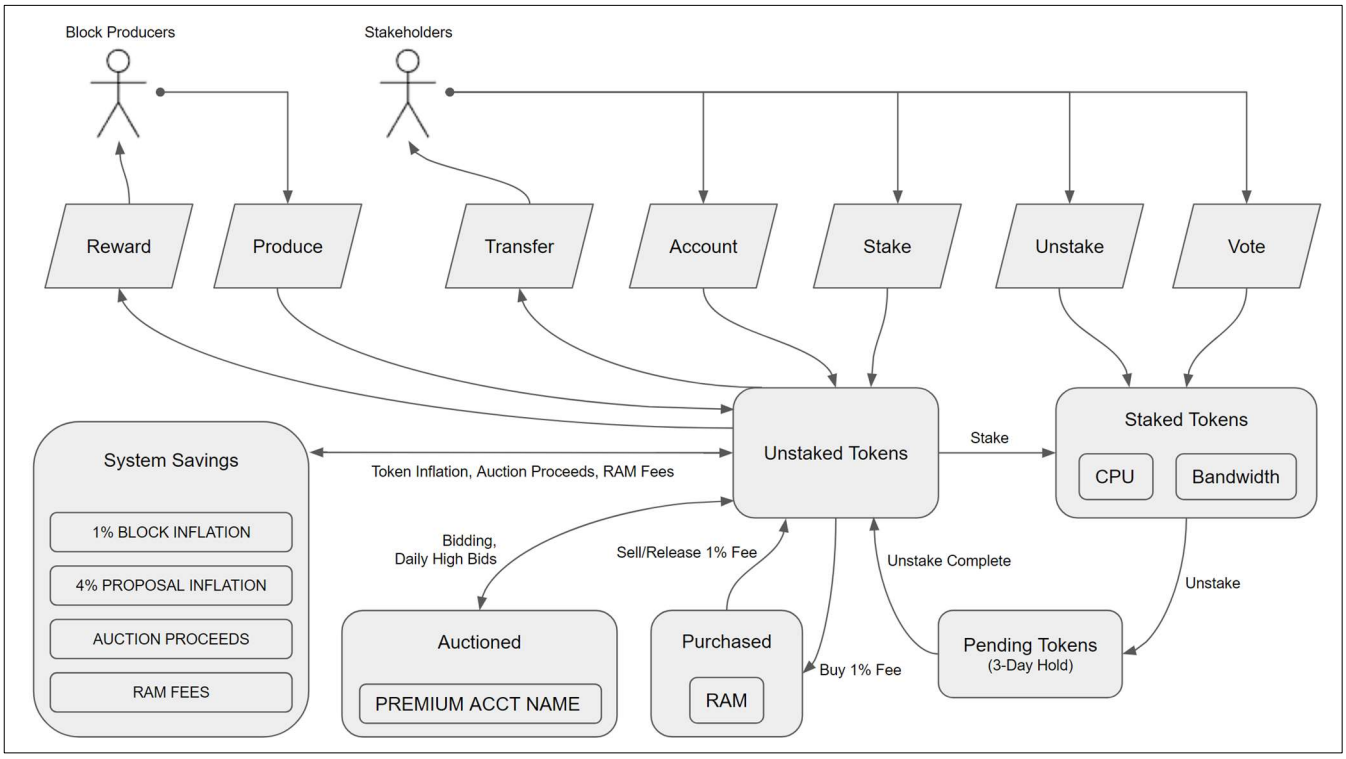
Các luồng OAuth phổ biến (Authorization Flows):

Luồng	Mô tả	Sử dụng cho
Authorization Code Flow	Người dùng xác thực → nhận mã code → app đổi mã lấy access token	Ứng dụng server-side
Implicit Flow *(deprecated)*	Token trả về trực tiếp qua URL	Ứng dụng frontend (SPA)
Client Credentials Flow	Client xác thực trực tiếp với Authorization Server	Machine-to-machine (M2M)
Resource Owner Password Flow *(không khuyến nghị)*	Người dùng cung cấp trực tiếp username/password cho app	Ứng dụng tin cậy cao
Device Authorization Flow	Dành cho thiết bị không có trình duyệt (TV, IoT)	Device-based apps

4.3.3 Quản lý vòng đời Token

Token là chuỗi đại diện quyền truy cập, được cấp bởi hệ thống ủy quyền (Authorization Server), dùng để chứng thực các yêu cầu truy cập tài nguyên. Có 2 loại phổ biến:

- **Access Token**: Dùng để truy cập API – thường có hạn sử dụng ngắn (5–15 phút).
- **Refresh Token**: Dùng để lấy access token mới – có thể có thời hạn dài hơn.



Hình 4.1. Vòng đời của Token. Nguồn: reddit.com/r/eos

4.4 THỰC HÀNH: JWTSECURE

4.4.1 Yêu cầu

Xây dựng bộ công cụ bảo mật JWT gồm các chức năng:

- *JWTValidator*: Kiểm tra toàn diện tính hợp lệ và độ an toàn của JWT.
- *JWTCracker*: Thử brute force các khóa bí mật yếu dùng để ký JWT.
- *OAuthClient*: Triển khai client OAuth 2.0 an toàn, tuân thủ các luồng xác thực chuẩn.
- *TokenManager*: Quản lý vòng đời JWT, bao gồm phát hành, làm mới và thu hồi refresh token.

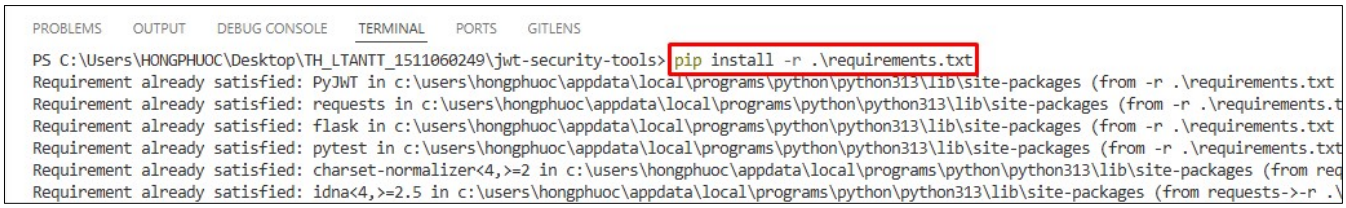
- *APIGateway*: Xây dựng cổng API đơn giản có tích hợp xác thực bằng JWT.

4.4.2 Hướng dẫn thực hành

- Tạo folder "`jwt-security-tools`". Trong folder này tạo các folder "`api_gateway`", "`jwt_cracker`", "`jwt_validator`", "`oauth_client`", "`tests`", "`token_manager`" và các file "`main.py`", "`pytest.ini`", "`requirements.txt`".
- Tạo file "`__init__.py`" trong các folder "`api_gateway`", "`jwt_cracker`", "`jwt_validator`", "`oauth_client`", "`tests`", "`token_manager`".
- Các gói cần thiết trong "`requirements.txt`":



- Cài đặt các gói cần thiết thông qua lệnh pip:



- File "`pytest.ini`":



- Code cho file "`api_gateway/gateway.py`":

jwt-security-tools > api_gateway > gateway.py > ...

```python
from flask import Flask, request, jsonify
import jwt

class APIGateway:
    def __init__(self, secret, algorithm="HS256"):
        self.secret = secret
        self.algorithm = algorithm
        self.app = Flask(__name__)
        self.routes()

    def routes(self):
        @self.app.route("/protected")
        def protected():
            token = request.headers.get("Authorization", "").replace("Bearer ", "")
            try:
                payload = jwt.decode(token, self.secret, algorithms=[self.algorithm])
                return jsonify({"message": "Access granted", "user": payload["sub"]})
            except jwt.InvalidTokenError:
                return jsonify({"message": "Access denied"}), 401

    def run(self):
        self.app.run(port=5000)

```

- Code cho file "`jwt_cracker/cracker.py`":

jwt-security-tools > jwt_cracker > cracker.py > ...

```python
import jwt
from concurrent.futures import ThreadPoolExecutor

class JWTCracker:
    def __init__(self, token, wordlist_path):
        self.token = token
        self.wordlist_path = wordlist_path

    def _try_secret(self, secret):
        try:
            jwt.decode(self.token, secret, algorithms=["HS256"])
            return secret
        except jwt.InvalidTokenError:
            return None

```

```
    def crack(self):
        with open(self.wordlist_path, "r") as f:
            secrets = f.read().splitlines()

        with ThreadPoolExecutor(max_workers=10) as executor:
            results = executor.map(self._try_secret, secrets)

        for result in results:
            if result:
                return result
        return None

```

- Code cho file "`jwt_validator/validator.py`":

jwt-security-tools > jwt_validator > validator.py > ...

```python
import jwt
from jwt.exceptions import InvalidTokenError, ExpiredSignatureError, InvalidSignatureError

class JWTValidator:
    def __init__(self, secret, algorithms=["HS256"]):
        self.secret = secret
        self.algorithms = algorithms

    def validate(self, token: str):
        try:
            payload = jwt.decode(token, self.secret, algorithms=self.algorithms)
            return {"valid": True, "payload": payload}
        except ExpiredSignatureError:
            return {"valid": False, "reason": "Token expired"}
        except InvalidSignatureError:
            return {"valid": False, "reason": "Invalid signature"}
        except InvalidTokenError as e:
            return {"valid": False, "reason": str(e)}

```

- Code cho file "`oauth_client/client.py`":

jwt-security-tools > oauth_client > client.py > ...

```python
import requests
from urllib.parse import urlencode

class OAuthClient:
    def __init__(self, client_id, client_secret, auth_url, token_url, redirect_uri):
        self.client_id = client_id
        self.client_secret = client_secret
        self.auth_url = auth_url
        self.token_url = token_url
        self.redirect_uri = redirect_uri
```

```
11
12      def get_auth_url(self, scope="openid"):
13          params = {
14              "response_type": "code",
15              "client_id": self.client_id,
16              "redirect_uri": self.redirect_uri,
17              "scope": scope
18          }
19          return f"{self.auth_url}?{urlencode(params)}"
20
21      def get_token(self, code):
22          data = {
23              "grant_type": "authorization_code",
24              "code": code,
25              "redirect_uri": self.redirect_uri,
26              "client_id": self.client_id,
27              "client_secret": self.client_secret
28          }
29          response = requests.post(self.token_url, data=data)
30          return response.json()
31
```

- Code cho file "`token_manager/manager.py`":

```
jwt-security-tools > token_manager > manager.py > ...
1   import jwt
2   import datetime
3
4   class TokenManager:
5       def __init__(self, secret, algorithm="HS256", access_exp=15, refresh_exp=60*24):
6           self.secret = secret
7           self.algorithm = algorithm
8           self.access_exp = access_exp
9           self.refresh_exp = refresh_exp
10
11      def _generate(self, payload, exp_minutes):
12          payload["exp"] = datetime.datetime.~~utcnow~~() + datetime.timedelta(minutes=exp_minutes)
13          return jwt.encode(payload, self.secret, algorithm=self.algorithm)
14
15      def generate_tokens(self, user_id):
16          access_token = self._generate({"sub": user_id, "type": "access"}, self.access_exp)
17          refresh_token = self._generate({"sub": user_id, "type": "refresh"}, self.refresh_exp)
18          return access_token, refresh_token
19
20      def refresh_access_token(self, refresh_token):
21          try:
22              payload = jwt.decode(refresh_token, self.secret, algorithms=[self.algorithm])
23              if payload.get("type") == "refresh":
24                  return self._generate({"sub": payload["sub"], "type": "access"}, self.access_exp)
25          except jwt.ExpiredSignatureError:
26              return None
27
```

- Code cho file "`main.py`":

jwt-security-tools > main.py > ...

```python
from api_gateway.gateway import APIGateway
from token_manager.manager import TokenManager

if __name__ == "__main__":
    secret = "supersecret"
    manager = TokenManager(secret)
    access_token, refresh_token = manager.generate_tokens("phuoc")

    print("Access Token:", access_token)
    print("Refresh Token:", refresh_token)

    gateway = APIGateway(secret)
    gateway.run()

```

- Code cho file "`tests/test_api_gateway.py`":

jwt-security-tools > tests > test_api_gateway.py > ...

```python
from api_gateway.gateway import APIGateway
from token_manager.manager import TokenManager
import pytest

@pytest.fixture
def client():
    secret = "mysecret"
    gateway = APIGateway(secret)
    gateway.app.config["TESTING"] = True
    return gateway.app.test_client(), TokenManager(secret)

def test_protected_route_with_valid_token(client):
    test_client, manager = client
    access_token, _ = manager.generate_tokens("tester")

    res = test_client.get("/protected", headers={"Authorization": f"Bearer {access_token}"})
    assert res.status_code == 200
    assert b"Access granted" in res.data

def test_protected_route_with_invalid_token(client):
    test_client, _ = client
    res = test_client.get("/protected", headers={"Authorization": "Bearer invalid"})
    assert res.status_code == 401

```

- Code cho file "`tests/test_cracker.py`":

jwt-security-tools > tests > test_cracker.py > ...

```python
from jwt_cracker.cracker import JWTCracker
import jwt
import tempfile

def test_crack_token():
    secret = "supersecret"
    token = jwt.encode({"sub": "test"}, secret, algorithm="HS256")

    with tempfile.NamedTemporaryFile(mode="w+", delete=False) as f:
        f.write("wrongpass\n123456\nsupersecret\n")
        f.seek(0)
        cracker = JWTCracker(token, f.name)
        found = cracker.crack()

    assert found == "supersecret"

```

- Code cho file "`tests/test_oauth_client.py`":

jwt-security-tools > tests > test_oauth_client.py > ...

```python
from oauth_client.client import OAuthClient

def test_auth_url():
    client = OAuthClient("client_id", "secret", "https://auth.com",
                         "https://token.com", "http://localhost")
    url = client.get_auth_url()
    assert url.startswith("https://auth.com?")
    assert "client_id=client_id" in url

```

- Code cho file "`tests/test_token_manager.py`":

jwt-security-tools > tests > test_token_manager.py > ...

```python
from token_manager.manager import TokenManager
import jwt

def test_token_creation_and_refresh():
    manager = TokenManager("key123")
    access, refresh = manager.generate_tokens("user1")

    payload = jwt.decode(access, "key123", algorithms=["HS256"])
    assert payload["sub"] == "user1"
    assert payload["type"] == "access"

    new_access = manager.refresh_access_token(refresh)
    assert new_access is not None

```

- Code cho file "`tests/test_validator.py`":

jwt-security-tools > tests > test_validator.py > ...

```python
from jwt_validator.validator import JWTValidator
import jwt
import datetime

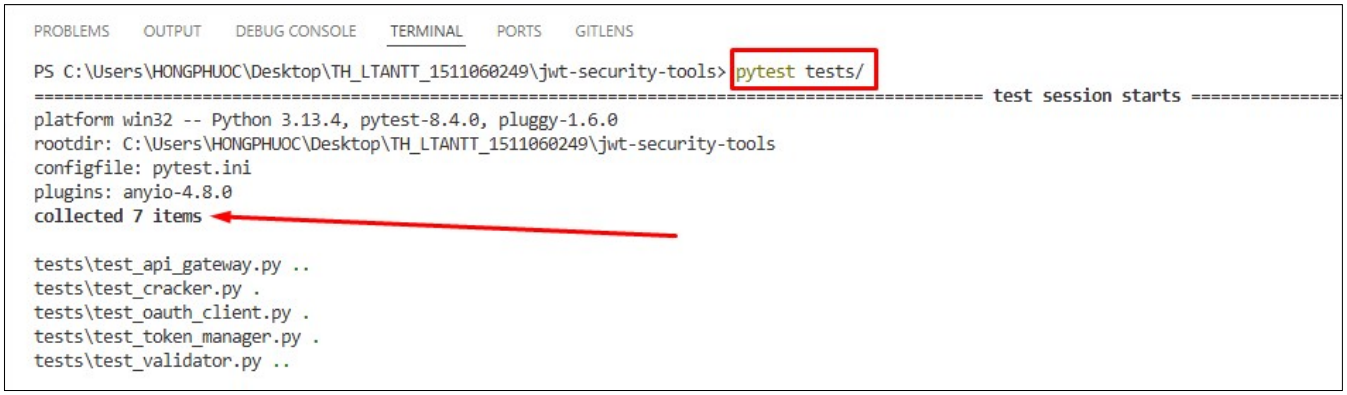
def generate_token(secret):
    payload = {"sub": "test", "exp": datetime.datetime.utcnow()
               + datetime.timedelta(minutes=5)}
    return jwt.encode(payload, secret, algorithm="HS256")

def test_valid_token():
    secret = "secret123"
    token = generate_token(secret)
    validator = JWTValidator(secret)
    result = validator.validate(token)
    assert result["valid"] is True
    assert result["payload"]["sub"] == "test"

def test_invalid_signature():
    token = generate_token("wrongsecret")
    validator = JWTValidator("correctsecret")
    result = validator.validate(token)
    assert result["valid"] is False

```

- Chạy pytest và kiểm tra:





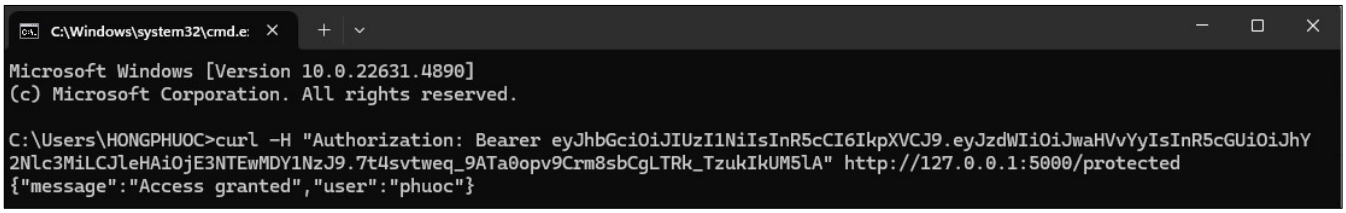
- Chạy "`main.py`" và lưu lại token:



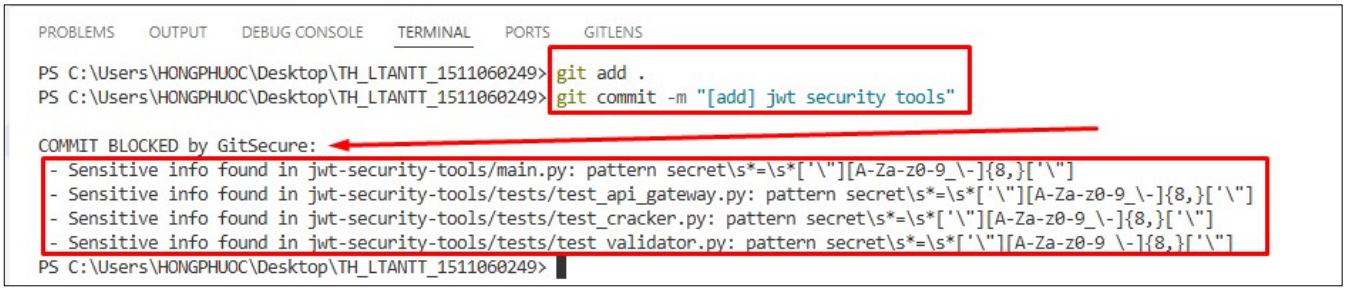
- Lưu lại Access Token và Refresh Token.

- curl -H "Authorization: Bearer <access_token>" http://127.0.0.1:5000/protected

- Kiểm tra chứng thực bằng cách vào Command Prompt:



- Commit code, push các thay đổi lên remote repo:



- Kiểm tra và xoá các thông tin nhạy cảm, sau đó tiến hành commit lại.

